

SecDash

Dashboard de Vulnerabilidades de Segurança Multi-Repositório

Gonçalo Filipe Domingos Carvalho
Rodrigo dos Ramos Pais Henriques Vitorino

Orientador: Professor José Simão

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

SecDash

49219 Gonalo Filipe Domingos Carvalho

49448 Rodrigo dos Ramos Pais Henriques Vitorino

Orientador: Professor Doutor Jos  Sim o

Relat rio do projeto realizado no  mbito de Projecto e Semin rio
Licenciatura em Engenharia Inform tica e de Computadores

Junho de 2026

Resumo

O **SecDash** é uma plataforma web de monitorização de relatórios de segurança que agrega e centraliza vulnerabilidades identificadas por ferramentas externas, nomeadamente GitHub Dependabot e Code Scanning e GitLab SAST e Dependency Scanning, em diferentes repositórios.

A aplicação pretende fornecer aos seus utilizadores uma visão unificada das análises de segurança efectuadas pelas ferramentas indicadas acima, eliminando a necessidade de consultar cada plataforma e respetivos repositórios individualmente.

O desenvolvimento deste projeto utilizou como base React e Typescript no frontend, Kotlin e Spring Boot no backend e PostgreSQL para a base de dados. A autenticação dos utilizadores é feita com base no protocolo OAuth2.0.

Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa, designadamente chatbots LLM como ChatGPT, Gemini e Claude Code para clarificação de conceitos teóricos otimização de sintaxe, bem como para auxílio na criação de componentes visuais. Para além disso, estas ferramentas foram empregues na geração de código ”*boilerplate*” e pesquisa de bibliotecas e técnicas previamente por nós desconhecidas, minimizando o tempo despendido na procura de soluções de implementação.

Índice

1	Introdução	1
2	Enquadramento e estado da arte	5
2.1	Static Application Security Testing e análise de dependências	5
2.1.1	Static Application Security Testing (SAST)	6
2.1.2	Análise de dependências	6
2.2	Estado da arte	7
2.2.1	DefectDojo	7
2.2.2	ArmorCode	8
3	Arquitetura e tecnologias	9
3.1	Requisitos funcionais	9
3.1.1	Integração de repositórios	9
3.1.2	Autenticação de utilizadores	9
3.1.3	Controlo de acesso com base em papéis	10
3.1.4	Agregação de vulnerabilidades	10
3.1.5	Dashboard Web	11
3.1.6	Exportação de relatórios	11
3.2	Objetivos opcionais	12
3.2.1	Integração com Jenkins	12
3.2.2	Deployment em contentores	12
3.2.3	Tracking de <i>Findings</i>	12
3.3	Visão geral do sistema	13
3.4	APIs externas	14
3.5	Tecnologias e linguagens	14
3.5.1	Backend	14
3.5.2	Frontend	17
3.6	Modelo de dados	18
4	Implementação	21
4.1	Backend	21

4.1.1	Controllers	22
4.1.2	Services	23
4.1.3	Repositories e REST Clients	24
4.1.4	Security Config	24
4.1.5	Normalização dos dados provenientes das APIs externas	24
4.1.6	Pedido e <i>scheduling</i> de <i>scans</i>	25
4.2	Frontend	26
4.2.1	<i>Internationalization</i> (i18n)	27
4.2.2	<i>Dashboard</i>	27
5	Conclusão	31
5.1	Reflexão sobre o desenvolvimento	31
5.2	Trabalho Futuro	32
	Referências	34
A	Exemplo do formato de resposta JSON de um repositório GitHub	35
B	Script SQL para a criação da base de dados	39

Capítulo 1

Introdução

O desenvolvimento de software moderno frequentemente utiliza bibliotecas *open source* para aumentar a velocidade de desenvolvimento e facilitar a implementação de componentes e funcionalidades comuns. A utilização destas bibliotecas de terceiros muitas vezes acaba por ser uma das características fundamentais do desenvolvimento de software atual, permitindo obter um menor *time-to-market* e uma melhor qualidade do sistema de software.[1]

Contudo, a utilização destes componentes de código aberto pode introduzir vulnerabilidades de segurança, o que tem resultado em diversos incidentes de alto perfil nos últimos anos [2]. De entre os exemplos mais críticos e mediáticos deste tipo de incidentes nos últimos anos contam-se o *Heartbleed*[3] ou o *Log4Shell*[4]. Este primeiro consistiu numa falha crítica na biblioteca OpenSSL, amplamente utilizada para implementar o protocolo HTTPS por vários websites. Este incidente foi particularmente grave uma vez que resultou na exposição de chaves privadas e outros dados de alta sensibilidade. Para além disso tratava-se de uma vulnerabilidade de exploração simples e que não deixava rasto, tendo o facto de ter permanecido indetetada durante um longo período amplificado significativamente o seu impacto global.

Por outro lado, o incidente *Log4Shell*, identificado no final de 2021, afetou a biblioteca de *logging* Java Apache Log4j. Esta vulnerabilidade foi classificada com a pontuação máxima de severidade (10.0 no sistema Common Vulnerability Scoring System (CVSS)), dado que permitia a execução remota de código de forma trivial através da manipulação de mensagens de log. Devido à integração desta biblioteca em milhares de serviços empresariais, a magnitude desta vulnerabilidade foi amplamente reconhecida pela indústria: a empresa de cibersegurança Tenable classificou-a como «a maior e mais crítica vulnerabilidade de sempre» [5], enquanto a publicação Ars Technica a descreveu como, «discutivelmente a vulnerabilidade mais grave de sempre» [6]. Por sua vez, o The Washington Post salientou que as descrições feitas por profissionais de cibersegurança sobre o impacto do incidente «roçam o apocalíptico» [7].

O caso da Equifax, empresa considerada um dos três maiores bureaus de crédito dos

Estados Unidos da América, ilustra também o problema. Uma auditoria interna em 2015 revelou sérios problemas de organização no seu departamento de TI. O relatório elaborado com base nesta auditoria concluiu que a organização não cumpria os seus próprios calendários de *patching*, que a equipa carecia de um inventário de recursos abrangente e que não existia uma priorização dos *patches* a serem efetuados com base na criticidade. Embora o relatório tenha delineado medidas para reforçar a segurança, muitas destas não foram implementadas atempadamente [8]. Dois anos mais tarde, em 2017, os sistemas informáticos da Equifax foram invadidos com recurso a uma vulnerabilidade na *framework* Apache Struts já conhecida e corrigida meses antes. O ataque, que podia ter sido facilmente evitado com uma melhor organização operacional, resultou na exposição de dados pessoais de mais de 150 milhões de pessoas, de entre os quais alguns de alta sensibilidade tais como números de segurança social e de cartões de crédito [9].

Ainda assim, o uso destas bibliotecas de código aberto tem vindo a crescer. Como tal, várias ferramentas de análise e *pipelines* CI/CD foram desenvolvidas. As equipas de desenvolvimento de software modernas dependem muitas vezes da integração destas ferramentas de *scanning* nas suas *pipelines* CI/CD tais como GitHub Dependabot, GitLab SAST, Trivy ou OWASP Dependency-Check. Apesar de estas ferramentas serem eficazes na deteção de vulnerabilidades conhecidas, os seus resultados ficam dispersos por diferentes repositórios e plataformas, tornando-se difícil obter uma visão unificada e consolidada da postura de segurança global de uma organização.

O **SecDash** pretende ser uma plataforma web concebida para agregar a visualização de vulnerabilidades de segurança já identificadas por estas ferramentas externas ou por *pipelines* de CI/CD existentes. Em vez de realizar a sua própria análise, o SecDash recolhe relatórios de vulnerabilidades de múltiplos repositórios de código-fonte e apresenta-os através de um *dashboard* único e interativo, permitindo que arquitetos de segurança de software e responsáveis de desenvolvimento monitorizem o risco em todos os seus projetos de forma imediata. O código da aplicação desenvolvida encontra-se disponível em <https://github.com/gfdcarvalho/secdash>.

No presente documento começaremos por, no **Capítulo 2** apresentar e analisar algumas das soluções já existentes no mercado que partilham objetivos semelhantes aos do **SecDash**, bem como explicar sucintamente os conceitos de *Static Application Security Testing* (SAST) e de análise de dependências, sobre os quais assentam as funcionalidades base da aplicação.

No **Capítulo 3** indicaremos brevemente as tecnologias e linguagens utilizadas no seu desenvolvimento e apresentaremos também um resumo simples da arquitetura que a sustenta.

No **Capítulo 4** descreveremos a implementação do **SecDash** de forma mais detalhada.

Finalmente, no **Capítulo 5**, concluiremos o presente documento partilhando as dificuldades técnicas encontradas durante a elaboração do projeto e iremos também expor algumas considerações finais.

Capítulo 2

Enquadramento e estado da arte

Perante o cenário histórico de vulnerabilidades críticas como as anteriormente mencionadas, a necessidade de ferramentas de agregação torna-se evidente, existindo atualmente várias soluções amplamente utilizadas na indústria. Contudo, estas plataformas distinguem-se do **SecDash** pelos seus objetivos e complexidade; focam-se habitualmente no segmento *enterprise* e na gestão operacional de ciclo completo, incluindo a orquestração de múltiplos *scanners*.

O **SecDash**, em contrapartida, não pretende ser uma ferramenta que efetua os seus próprios *scans* de segurança, mas sim focar-se especificamente na agregação, normalização e visualização centralizada de vulnerabilidades provenientes de múltiplos repositórios em plataformas distintas. O objetivo principal do projeto consiste em fornecer uma integração totalmente funcional com o **GitHub** e o **GitLab**, as duas plataformas mais amplamente utilizadas para hospedar código, uma vez que estas abrangem a vasta maioria dos casos de uso em ambientes reais. A integração com *pipelines* de **Jenkins** é considerada um objetivo opcional, a ser explorado caso a calendarização assim o permita.

2.1 Static Application Security Testing e análise de dependências

Para o desenvolvimento de uma aplicação que pretende agregar resultados de análises de segurança de código-fonte é essencial compreender as metodologias que originam esses resultados. Estas podem ser divididas em duas categorias: a análise estática de código próprio e a auditoria de código externo proveniente de bibliotecas e outras dependências.

Enquanto o **Static Application Security Testing (SAST)** foca a sua análise no código proprietário desenvolvido internamente, a análise de dependências foca-se no código externo proveniente de bibliotecas de terceiros integradas no projeto.

2.1.1 Static Application Security Testing (SAST)

Static Application Security Testing (SAST), em suma, refere-se à análise de código-fonte ou código binário de um programa sem que este se encontre em execução. Deste modo é possível identificar várias vulnerabilidades de segurança no código de uma aplicação ao longo do seu processo de desenvolvimento, permitindo a deteção de problemas como vulnerabilidades a SQL injections ou credenciais como chaves de API embutidas no código desde muito cedo, quando estes ainda são fáceis e pouco dispendiosos de corrigir [10] [11].

Atualmente, tanto o GitHub como o GitLab integram ferramentas nativas de SAST nos seus ecossistemas, automatizando a execução destas análises através dos seus respetivos pipelines de integração contínua (CI/CD).

No GitHub, a funcionalidade de SAST está integrada no ecossistema de **Code Scanning** e assenta principalmente no motor **CodeQL**.

Por sua vez, no GitLab, o processo é designado nativamente por **GitLab SAST**. Ao contrário do GitHub, que centraliza a análise num único motor proprietário, o CodeQL, o GitLab adota uma estratégia de orquestração de múltiplos scanners *open-source*.

2.1.2 Análise de dependências

Ao contrário do SAST, que avalia o código desenvolvido de raiz, a análise de dependências foca-se na cadeia de abastecimento de software (*software supply chain*).

O processo passa por analisar os manifestos de gestão de dependências do projeto, como os ficheiros `build.gradle.kts` ou `package.json` para extrair a lista e as versões exatas de todas as bibliotecas utilizadas, incluindo as dependências transitivas (isto é, as dependências das dependências). Atualmente o GitHub e o GitLab automatizam este processo recorrendo a mecanismos próprios.

No GitHub este tem o nome **Dependabot** e utiliza a **GitHub Advisory Database**, uma base de dados que agrega dados de fontes públicas (como o catálogo norte-americano *National Vulnerability Database*) e de alertas submetidos pela comunidade.

Por sua vez, no GitLab, o processo é designado nativamente por **Dependency Scanning**.

2.2 Estado da arte

De entre programas e aplicações com intuitos semelhantes ao do **SecDash** destacamos, entre outros, o **DefectDojo** [12] e **ArmorCode** [13].

2.2.1 DefectDojo

O **DefectDojo** é uma plataforma DevSecOps que atua como um agregador automático para o seu conjunto de ferramentas de segurança, permitindo organizar facilmente o trabalho de segurança e reportar a postura de segurança da organização aos *stakeholders*. [14]

De entre as funcionalidades do DefectDojo contam-se, entre outras:

- O rastreio e notificação de *Findings* de segurança;
- A imposição de SLAs (*Service Level Agreement*) em contexto;
- A gestão de *risk-acceptance* e outras decisões de triagem
- Coordenação com gestão de testes de penetração tradicionais;

Para além de possibilitar a integração com mais de 200 ferramentas de segurança, a sua instalação local requer múltiplos contentores para vários serviços, requerindo inclusive instâncias dedicadas de *Celery* e *Redis* enquanto *task-queue* para a gestão de tarefas assíncronas tais como o processamento de ficheiros de notificações volumosos ou a sincronização com o *Jira* e envio de notificações aos utilizadores. Estamos portanto perante uma infraestrutura comparativamente mais complexa que a requirida pelo **SecDash**, que será mais apropriada para equipas pequenas que não requeiram todas as funcionalidades disponíveis no DefectDojo.

Por outro lado, o modelo de dados utilizado pelo **DefectDojo** utiliza cinco classes distintas (***Product Types***, ***Products***, ***Engagements***, ***Tests*** e ***Findings***) para organizar o trabalho das equipas que o utilizem. [15] Um Engagement, por exemplo, representa um momento no tempo em que um *Test* é realizado, e contem um ou mais *Tests*. A necessidade de instanciar um *Engagement* para cada nova análise introduz uma barreira de agilidade significativa. Apesar de se apresentar como flexível, de forma a que este modelo se possa adaptar às diferentes equipas que o utilizem, a abordagem do DefectDojo revela uma preocupação mais voltada para questões de auditoria do que para a visão imediata de vulnerabilidades e agilidade durante o processo de desenvolvimento do software.

2.2.2 ArmorCode

O **ArmorCode** é uma aplicação comercial *enterprise* de *Application Security Posture Management* (ASPM) que oferece integração com mais de 350 ferramentas de segurança de aplicações, infraestrutura, cloud e contentores, bem como com sistemas de DevOps.

De entre as suas vastas funcionalidades faz também parte a agregação de vulnerabilidades encontradas por outras ferramentas de *scanning*. De forma semelhante ao **SecDash**, o ArmorCode não faz ele próprio um scan de vulnerabilidades, apresentando-se como uma solução «*scannerless, vendor-agnostic*» [16]. Mais uma vez, no entanto, estamos perante uma ferramenta de elevada complexidade, possuindo imensas funcionalidades para lá do escopo do problema que o **SecDash** pretende resolver e que para além disso podem levar a uma curva de aprendizagem acentuada que poderá ser contraproducente quando se procura uma solução ágil para a simples agregação de relatórios de vulnerabilidades externos.

Quando comparado com o DefectDojo temos ainda a agravante de esta ser uma ferramenta SaaS proprietária com um modelo de licenciamento orientado para grandes corporações, o que a torna inacessível para projetos independentes ou de equipas de pequena dimensão.

Capítulo 3

Arquitetura e tecnologias

O presente capítulo pretende expor uma visão geral dos requisitos da aplicação e da organização técnica do projeto, os diferentes módulos que o compõem e suas interdependências, bem como as linguagem de programação e *frameworks* usadas no seu desenvolvimento.

3.1 Requisitos funcionais

De forma a responder satisfatoriamente ao problema que o **SecDash** pretende resolver, foi definida uma lista de requisitos funcionais considerados essenciais para o sistema. Estes requisitos representam as funcionalidades mínimas necessárias para permitir a agregação, normalização e visualização centralizada de vulnerabilidades provenientes de múltiplos repositórios e plataformas.

3.1.1 Integração de repositórios

A aplicação deve ser capaz de obter repositórios alojados nas plataformas GitHub e GitLab através das respetivas APIs.

Após a autenticação do utilizador e obtenção das permissões necessárias, o sistema deverá conseguir:

- Obter a lista de repositórios associados ao utilizador;
- Persistir a informação dos repositórios a ser vigiados pela aplicação numa base de dados;
- Associar os repositórios ao utilizador autenticado;

3.1.2 Autenticação de utilizadores

A plataforma deve disponibilizar mecanismos de autenticação que permitam aos utilizadores vigiar de forma persistente os seus repositórios escolhidos garantindo ao mesmo tempo a sua

confidencialidade.

Para isso deverá existir suporte à autenticação através do protocolo OAuth2, recorrendo aos serviços disponibilizados pelo GitHub e GitLab. Este mecanismo permitirá:

- Autenticar utilizadores sem necessidade de credenciais locais;
- Garantir o acesso autenticado às APIs externa, obtendo assim permissão para acesso aos repositórios privados de um utilizador;
- Simplificar o processo de registo e login.

Após autenticação bem sucedida, o utilizador deverá poder consultar os seus repositórios previamente guardados sem necessidade de repetir o processo de autorização às plataformas externas.

Adicionalmente, o registo e autenticação no **SecDash** pode também ser feito através de e-mail e password.

3.1.3 Controlo de acesso com base em papéis

O sistema deverá implementar um mecanismo de controlo de acesso baseados em papéis (**Role-Based Access Control** ou **RBAC**), permitindo restringir funcionalidades de acordo com o tipo de utilizador. Pretende-se assim que os responsáveis pela manutenção do serviço possam ter total controlo pela gestão da aplicação.

Deverão ser suportados os seguintes papéis:

- **ADMIN**: Utilizador com funções de administração. Pode adicionar e remover utilizadores e repositórios de equipas às quais não pertence, eliminar equipas que não lidera.
- **USER**: Utilizador comum da aplicação.

Para além destes roles, geridos pelo Spring Security, dentro de cada equipa um utilizador irá ter também um de dois papéis: **Leader** ou **Collaborator**. No entanto, este controlo é feito apenas ao nível dos serviços.

3.1.4 Agregação de vulnerabilidades

Uma das funcionalidades mais importantes do SecDash deverá ser a capacidade de recolher e consolidar vulnerabilidades provenientes das análises feitas pelo GitHub Dependabot, GitLab

Dependency Scanning e relatórios SAST de ambas as plataformas.

A informação obtida deverá ser convertida para um modelo de dados interno normalizado, independentemente do formato original utilizado pela plataforma de origem. O sistema deverá então:

- Importar relatórios de vulnerabilidades de ambas as plataformas GitHub e GitLab;
- Identificar e exibir as vulnerabilidades associadas a cada repositório;
- Normalizar os dados heterogêneos provenientes de cada plataforma externa;
- Permitir a consulta agregada e centralizada dos relatórios e findings provenientes de cada plataforma externa.

Este requisito constitui um dos principais desafios técnicos do projeto devido à heterogeneidade dos formatos produzidos pelas diferentes ferramentas de cada plataforma.

3.1.5 Dashboard Web

A plataforma deverá disponibilizar uma interface web que permita aos utilizadores visualizar o estado de segurança dos repositórios monitorizados.

O dashboard deverá incluir:

- Listagem de vulnerabilidades;
- Listagem de SAST Alerts;
- Gráficos de visualização da evolução de alertas de segurança ao longo do tempo.

Pretende-se que a interface seja visualmente clara e de fácil acessibilidade, permitindo-lhes identificar rapidamente problemas de segurança relevantes identificados pelas ferramentas de segurança da plataforma originária de cada repositório.

3.1.6 Exportação de relatórios

O sistema deverá permitir exportar relatórios de vulnerabilidades para formatos externos, possibilitando a sua partilha e integração com outros sistemas.

Inicialmente, deverá ser suportada exportação em formato CSV, podendo futuramente ser adicionados outros formatos.

Os relatórios exportados deverão incluir:

- Informação do(s) repositório(s) analisado(s);
- Resumo das vulnerabilidades identificadas;
- Data de geração do relatório.

3.2 Objetivos opcionais

No caso de a calendarização o permitir, pretendem-se implementar os seguintes objetivos opcionais. Consideramos que estes objetivos poderiam acrescentar mais alguma usabilidade à aplicação, embora não sejam estritamente necessários para responder ao problema que esta pretende desenvolver.

3.2.1 Integração com Jenkins

Pretende-se acrescentar fonte de dados adicional para além das plataformas GitHub e GitLab, integrando o **SecDash** com o Jenkins, um dos servidores de automação mais utilizados na indústria para CI/CD.

3.2.2 Deployment em contentores

De forma a validar a viabilidade operacional da plataforma, pretende-se realizar o *deployment* do **SecDash** alojando-o num servidor pessoal.

Este processo envolveu a configuração e gestão de redes para exposição controlada do serviço e a orquestração via Docker. O objetivo é que a aplicação seja demonstrada num ambiente que mais se assemelha a um ambiente real e não apenas no ambiente de desenvolvimento local, passando a aplicação a estar acessível de forma remota.

Pretende-se deste modo alargar o âmbito do projeto para lá de apenas desenvolvimento de software, passando pela gestão de regras de *firewall* e exposição segura de serviços para acesso remoto.

3.2.3 Tracking de *Findings*

Este objetivo visa dotar o **SecDash** de capacidades *tracking* sobre as vulnerabilidades detetadas.

Pretende-se implementar um sistema de acompanhamento que permita verificar se determinada vulnerabilidade já se encontra ao encargo de algum desenvolvedor da equipa ou se

esta já se encontra resolvida e, em caso afirmativo, desde quando.

3.3 Visão geral do sistema

O projeto encontra-se dividido em dois blocos principais, *frontend* e *backend*, interligados entre si.

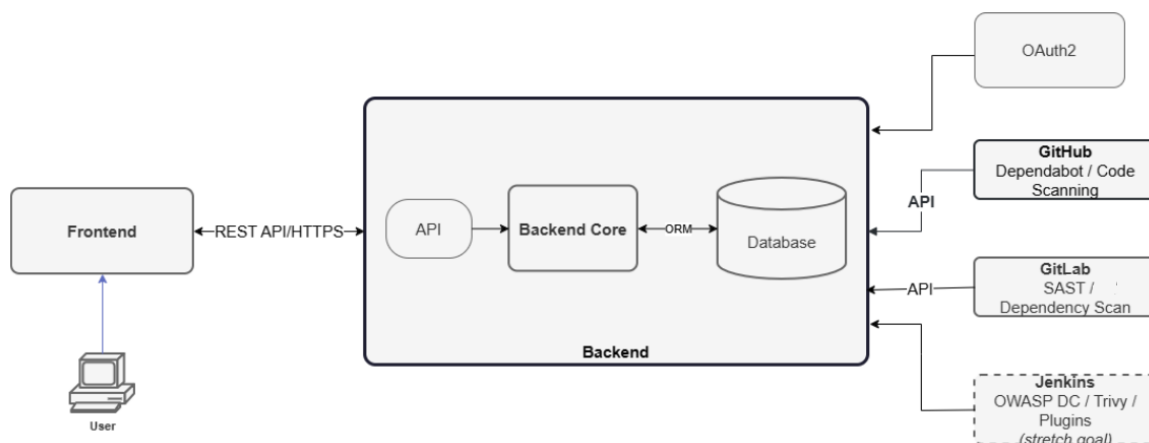


Figura 3.1: Diagrama da arquitetura mostrando a separação das diferentes camadas da aplicação

O *backend* é o principal responsável pela lógica de negócio da aplicação, tendo a responsabilidade de comunicar com as APIs externas às quais a aplicação necessita de recorrer e de normalizar a informação recebida destas. É também sua responsabilidade toda a lógica de registo e autorização dos utilizadores e a persistências dos dados numa base de dados. O *backend* atua também como uma **API REST**, expondo os dados necessários de forma estruturada via pedidos **HTTP**.

Enquanto isso, o *frontend* constitui a interface visual com que o utilizador interage via web browser. O seu papel é consumir os dados enviados pelo *backend* e exibí-los ao utilizador de forma interativa através de uma página web, tratando de toda a lógica de navegação e visualização.

Esta separação modular, tendo por base o princípio de *separation of concerns*, para além de permitir escolhas técnicas mais especializadas para cada módulo, reduz a complexidade ao limitar a interação entre estes, permitindo que cada camada evolua de forma independente, facilitando a manutenção e atualização futura do sistema. Para além disso, separar a lógica de apresentação da lógica de dados permite que uma única API, implementada no *backend*

venha a servir potenciais múltiplas plataformas (web, desktop, mobile, etc).

3.4 APIs externas

O funcionamento da aplicação está intrinsecamente ligado ao uso de APIs externas. Os utilizadores podem-se autenticar através da utilização do protocolo OAuth 2.0 usando como *identity providers* Google, GitHub ou GitLab.

Estes dois últimos provedores de identidade cumprem um papel duplo, pois para além da possibilidade de providenciarem *autenticação* ao **SecDash** são também eles que providenciam *autorização*, de forma a que a aplicação possa requisitar os repositórios dos utilizadores bem como consumir os endpoints de segurança que contêm os relatórios de vulnerabilidades sobre os quais a funcionalidade base do projeto assenta.

3.5 Tecnologias e linguagens

3.5.1 Backend

Kotlin

A decisão de escolher Kotlin como linguagem utilizada para o desenvolvimento do *backend* foi tomada, em parte, devido à proficiência nela adquirida ao longo do percurso da licenciatura. Uma vez que foi a linguagem com maior exposição e utilização prática em mais unidades curriculares ao longo do percurso académico deste ciclo de estudos, esta escolha permitiu reduzir a curva de aprendizagem necessária à elaboração do projeto, acelerando assim o seu desenvolvimento.

Para além disso, a linguagem Kotlin possui uma série de características que auxiliam o desenvolvimento e fortalecem a robustez da aplicação, de entre as quais se destacam duas:

- **Segurança de tipos e *Null Safety*:** O sistema de tipos do Kotlin ajuda a prevenir erros comuns de programação, tais como *NullPointerExceptions*, garantindo que a manipulação de dados sensíveis provenientes das APIs externas seja feita de forma segura;
- **Interoperabilidade com o ecossistema Java Virtual Machine (JVM):** O acesso às bibliotecas e frameworks da JVM não só garante que o sistema assenta numa base tecnológica estável como permite usar ferramentas como Spring Boot ou JDBI, agilizando o desenvolvimento da API e ligação à base de dados;

Spring Boot

A framework Spring Boot constitui a base fundamental do *backend*. A sua filosofia "*convention-over-configuration*" [17], permite minimizar a preocupação com questões de configuração para que o foco do desenvolvimento possa incidir apenas na lógica aplicacional, tornando o processo de programação mais célere.

O Spring Boot é particularmente útil pelos seguintes pontos:

- ***Inversion of Control (IoC)* e Injeção de Dependências:** O Spring gere automaticamente o ciclo de vida dos componentes da aplicação. No contexto do **SecDash**, isto permite, por exemplo, que os clientes REST de integração com o GitHub e GitLab sejam injetados nos serviços correspondentes de forma transparente;
- **Segurança com Spring Security:** A dependência Spring Security facilita a implementação de fluxos de autenticação OAuth2 e a proteção de determinados *endpoints* através de configurações de autorização granulares, garantindo que dados sensíveis e algumas funcionalidades da aplicação são acedidos apenas por utilizadores com as permissões adequadas;
- **Servidor embutido:** O Spring Boot permite que a aplicação seja compilada para um único ficheiro executável que inclui um servidor Apache Tomcat embutido. Para além de eliminar a necessidade de configurar um servidor externo, isto poderá facilitar a contentorização da aplicação através da plataforma Docker.

JDBI

Para a camada de interação com a base de dados, optou-se pela utilização da biblioteca **Jdbi**. Esta biblioteca segue uma filosofia "*SQL-centric*", conferindo algumas vantagens de conveniência quando comparada com a biblioteca (*Java Database Connectivity (JDBC)*) sobre a qual esta assenta.

Esta apresenta quatro grandes vantagens sobre o nosso ponto de vista:

- **Controlo total sobre o SQL:** A JDBI permite escrever manualmente as consultas SQL a serem executadas. Este controlo garante o desempenho e a correção de consultas complexas;
- **Mapeamento Nativo para *Data Classes*:** A integração com o Kotlin permite que os resultados das consultas SQL sejam mapeados automaticamente para *Data Classes*.

- **Facilidade de *debugging*:** O comportamento da JDBI é inteiramente previsível. Em caso de erro, este é rastreável diretamente até ao SQL executado, simplificando o processo de debugging durante o desenvolvimento;
- **Eliminação de código repetitivo e gestão automática de recursos:** Em vez de obrigar o programador a gerir manualmente a abertura e fecho de ligações, a criação de *statements* ou a iteração sobre *result sets*, o Jdbi automatiza estas tarefas. O Jdbi garante assim que todos os recursos da base de dados (ligações, *statements* e *result sets*) são libertados corretamente após a execução, mesmo que esta termine em erro. O uso de JDBC puro, comparativamente, requeriria que este tratamento fosse feito de forma explícita em todos pedidos à base de dados;

PostgreSQL

Para persistência de dados decidiu-se utilizar uma base de dados relacional, uma vez que os dados com os quais a aplicação trabalha são altamente estruturados e com dependências e relações claras entre si (por exemplo, um utilizador possui múltiplos repositórios associados e cada repositório agrupa um conjunto delimitado de vulnerabilidades). O **PostgreSQL** permite mapear este modelo de dados de forma direta, utilizando chaves primárias e estrangeiras para refletir fielmente a lógica de negócio. Este fator, aliado à simplicidade no mapeamento com a Jdbi e integração com o ecossistema Kotlin/JVM, levaram à escolha de PostgreSQL.

OAuth 2.0

A utilização do protocolo **OAuth 2.0** no **SecDash** cumpre um propósito duplo.

Um destes propósitos é a delegação da gestão de identidades a entidades externas confiáveis, não sendo assim necessário que um utilizador se registe diretamente no **SecDash**. Os utilizadores podem assim criar conta e aceder à plataforma com apenas um clique, utilizando contas já existentes em outras plataformas, o que pode melhorar significativamente a experiência de utilização. As plataformas suportadas como provedores de identidade são Google, GitHub e GitLab.

Para além disso, a funcionalidade principal da aplicação está dependente deste protocolo: sendo o **SecDash** um agregador de vulnerabilidades, a aplicação necessita de consultar dados privados guardados no GitHub e GitLab, sendo o OAuth2.0 a forma como o acesso a estes é concedido.

3.5.2 Frontend

Typescript

A linguagem escolhida para o *frontend* foi **Typescript**. A escolha desta em detrimento de Javascript puro deveu-se ao facto de Typescript possuir tipagem estática. Não só isto facilita a transposição dos modelos de dados utilizados no *backend* para a lógica do frontend como também garante que qualquer erro de correspondência de dados possa ser detetado imediatamente em tempo de compilação, e não em tempo de execução no navegador do utilizador.

React

Como biblioteca principal para a construção da interface do utilizador, adotou-se **React**.

Os fatores que motivaram esta escolha foram os seguintes:

- **Arquitetura Baseada em Componentes:** O React permite decompor a interface visual em peças de código modulares, isoladas e reutilizáveis. Esta modularidade facilita a manutenção do código e permite que múltiplos elementos partilhem a mesma lógica visual e a sua reutilização;
- **Gestão de Estado com *Hooks*:** A utilização de *Hooks* como `useState` e `useEffect` permitem controlar o fluxo de dados na aplicação e gerir operações como o estado de carregamento de componentes, por exemplo;

Vite

Como *module bundler* optou-se pela utilização de **Vite** em alternativa a Webpack, por ser o standard *de facto* para projetos em React. O Vite utiliza *ES Modules* nativos do browser ao correr o servidor de desenvolvimento em vez de fazer bundle de todo o código, tornando o seu arranque e *hot reloading* mais rápido.

Organização dos componentes

Sendo o *frontend* construído com React, toda a interface é composta por **componentes** organizados de forma hierárquica, formando aquilo a que se chama uma **árvore de componentes**. Nesta árvore, cada componente pode conter (renderizar) outros componentes, existindo um componente raiz (**App**) a partir do qual toda a interface é derivada.

A aplicação encontra-se estruturada em três grandes grupos: os ***providers***, que envolvem toda a aplicação e disponibilizam estado global (tema visual, autenticação, traduções e

navegação); as *views*, correspondentes às páginas associadas a cada rota; e os **componentes reutilizáveis**, peças de menor dimensão compartilhadas por várias *views*. Todas as páginas autenticadas são renderizadas dentro de um componente **Layout** comum, que fornece a barra de topo e o menu de navegação lateral.

A Figura 3.2 apresenta, de forma simplificada, a organização hierárquica dos principais componentes da aplicação.

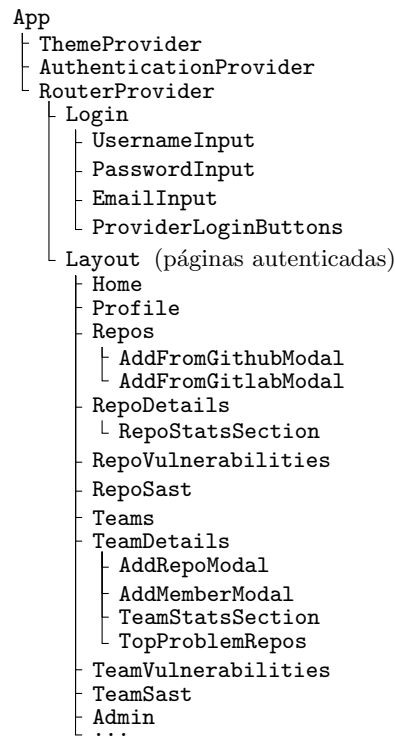


Figura 3.2: Árvore de componentes simplificada do *frontend*, ilustrando a hierarquia entre *providers*, o *layout* comum, as *views* de cada rota e os componentes reutilizáveis.

3.6 Modelo de dados

A elaboração de um modelo Entidade-Associação é indispensável para a estruturação lógica do projeto. Com efeito, é a existência de um modelo entidade-associação que permite estruturar de forma concreta as entidades do domínio da aplicação e as suas relações entre si.

O modelo de dados serve também de ponte entre os requisitos de negócio da plataforma e a sua implementação física na base de dados. Este mapeamento garante que a existência uma visão clara de como os utilizadores, repositórios e vulnerabilidades se relacionam entre si, permitindo orientar a implementação lógica do sistema prevenindo assim o aparecimento de inconsistências estruturais que pudessem pôr em causa a robustez da aplicação ou atrasar o seu desenvolvimento.

Como referido no capítulo anterior, os dados obtidos das APIs externas do GitHub e GitLab são altamente estruturados e com dependências e relações claras entre si. Assim sendo, a elaboração de um modelo Entidade-Associação torna-se relativamente trivial.

O diagrama do modelo obtido encontra-se apresentado na Figura 3.3. É importante salientar que, de forma a facilitar a apresentação e visualização do modelo no presente documento, os atributos das entidades, bem como a cardinalidade das suas relações, foram omitidos do diagrama. O script SQL utilizado para criar todo o esquema da base de dados, onde se pode consultar todos os atributos, encontra-se no Apêndice B.

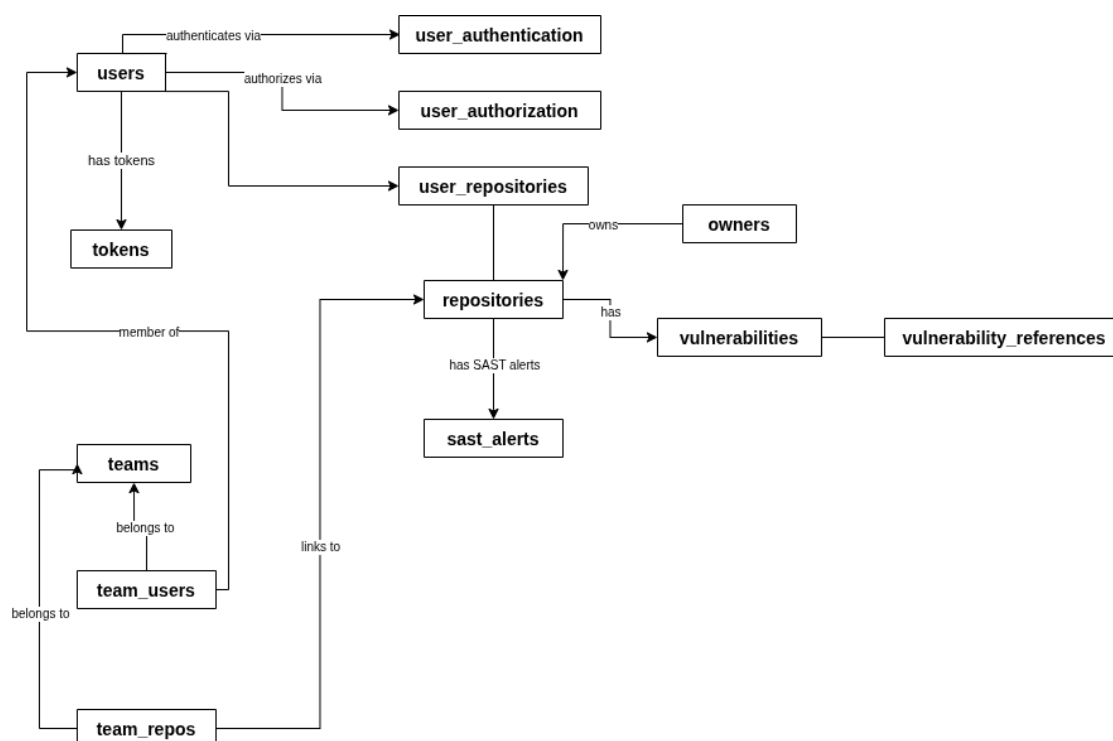


Figura 3.3: Diagrama do modelo de dados ilustrando sucintamente as ligações entre cada uma das entidades que o constituem

- Cada **USER** possui um **id** que os identificará na aplicação, bem como um **name** e **email**. Poderão também ter um campo ***password_validation***, opcional, caso optem por se registar na aplicação diretamente com o seu email em vez de utilizarem uma das opções OAuth 2.0.

Cada user terá também uma ***role***, que o identificará como um utilizador normal ou administrador, tendo este último acesso a funcionalidades vedadas a utilizadores normais,

tais como eliminar utilizadores ou equipas que não lhe pertencem.

As informações de autenticação são guardadas na tabela **USER_AUTHENTICATION**, que irá conter os IDs de cada utilizador, o provedor de autenticação OAuth 2.0 (Google, GitHub ou GitLab) e o ID do utilizador nessa plataforma.

Os *access tokens* de cada utilizador ficam armazenados na tabela **USER_AUTHORIZATION**. Os *tokens* internos gerados e usados pelo próprio **SecDash**, por sua vez, são armazenados na tabela **TOKENS**.

- Cada repositório vigiado pela aplicação é armazenado na tabela **REPOSITORIES**. A tabela **OWNERS** armazena algumas informações referentes aos proprietários dos repositórios nas plataformas de origem, seja ela GitHub ou GitLab. Para fazer a ligação entre os repositórios armazenados e os utilizadores que os vigiam existe a tabela **USER_REPOSITORIES**.

Os alertas SAST e vulnerabilidades detetadas nas dependências de cada repositório são armazenados nas tabelas **SAST_ALERTS** e **VULNERABILITIES** respetivamente. Esta última tabela encontra-se também ligada à tabela **VULNERABILITY_REFERENCES**, que armazena ligações as CVEs onde essas vulnerabilidades estão catalogadas.

- Finalmente, as informações de cada equipa criada no âmbito da aplicação são armazenadas na tabela **TEAMS**. Os repositórios e utilizadores ligados a cada uma são armazenados nas tabelas **TEAM_REPOS** e **TEAM_USERS** respetivamente.

É importante indicar que ao obter os dados das APIs externas, sejam estes referentes a repositórios ou vulnerabilidades, a informação devolvida contém inúmeros campos que não consideramos relevantes de armazenar ou exibir aos utilizadores para os propósitos do **SecDash**. A título de exemplo, o objeto JSON referente a um repositório GitHub devolvido pela sua API contém mais de 80 campos, sendo vários deles objetos que por sua vez contém também múltiplos campos. Um exemplo deste formato pode ser consultado no Apêndice A. Como tal, optou-se por filtrar muitos destes campos, sendo apenas armazenados aqueles que considerámos relevantes para o propósito da aplicação.

Capítulo 4

Implementação

Para facilitar a manutenção e implementação do sistema, bem como a sua compreensão, o **SecDash** encontra-se decomposto em camadas, seguindo um padrão de desenho *multi-tier* ou *layered*.

A aplicação encontra-se dividida em três níveis, numa estrutura hierárquica, sendo estes a **camada de apresentação**, a **camada lógica** e a **camada de dados**.

- A **camada de apresentação** tem como função principal ser a interface com a qual o utilizador interage com o **SecDash**, isto é, o nosso *frontend* web;
- A **camada lógica** é o cerne do programa, tratando-se do nosso backend em kotlin onde é implementada toda a lógica de negócio;
- A **camada de dados** persiste os dados a serem armazenados permanentemente, estando esta implementada como uma base de dados PostgreSQL;

Os detalhes e especificidades da implementação e desenvolvimento de cada uma destas camadas serão explorados ao longo deste capítulo.

4.1 Backend

O *backend* do **SecDash** encontra-se estruturado com o padrão de desenho **Controller-Service-Repository (CSR)**. Pretende-se deste modo isolar o tratamento de pedidos HTTP, a lógica de negócio e operações de base de dados em componentes independentes. Organizar a aplicação desta forma tem o intuito de facilitar o seu desenvolvimento e manutenção.

4.1.1 Controllers

A camada dos *controllers* tem como principal objetivo receber pedidos HTTP para consequentemente devolver respostas HTTP. A camada conta com nove controllers distintos, cada um com endpoints próprios para cada operação suportada pela aplicação:

- **Auth Controller:** Responsável por receber pedidos HTTP relacionados com autorização externa, tais como pedidos de *login* através de OAuth 2.0 na aplicação ou conceder acesso aos repositórios de um utilizador de uma das plataformas externas;
- **GitHub Controller:** Responsável por receber pedidos e emitir as respetivas respostas para tarefas que requeiram acesso ao GitHub, tais como adicionar um repositório do GitHub aos repositórios vigiados pelo utilizador com sessão ativa, ou obter análises de vulnerabilidade de um desses repositórios;
- **GitLab Controller:** Análogo ao controller anterior mas para GitLab;
- **Repo Controller:** Responsável por receber e responder a pedidos HTTP relacionados com os repositórios inseridos no domínio do **SecDash**;
- **Team Controller:** Responsável por receber pedidos HTTP e emitir as respetivas respostas para operações relacionadas com equipas no **SecDash**. Os utilizadores podem criar ou eliminar equipas, adicionar ou remover repositórios e outros utilizadores a equipas ou promover membros de uma equipa a Team Leader;
- **User Controller:** Responsável por receber e responder a pedidos HTTP para registo na aplicação através do formulário de registo ou de obter informação do utilizador com sessão ativa;
- **Vulnerability Controller:** Responsável por obter as vulnerabilidades já registadas no domínio da aplicação;
- **Sast Controller:** Responsável por obter os alertas Sast já registados no domínio da aplicação;
- **Admin Controller:** Responsável por receber e atender pedidos referentes a tarefas de administração da aplicação. Os endpoints deste controller apenas são acessíveis a utilizadores com a role de ADMIN;

Para manter o isolamento das camadas, nenhuma lógica de negócio ou processamento de dados é executado nesta camada. Esse tipo de tarefas é delegado para a camada dos serviços, que será detalhada na subseção seguinte. Não obstante, para efeitos de organização do código, optou-se por colocar alguns componentes necessários para uma utilização mais eficiente da

framework Spring e evitar repetição de código numa subpackage intitulada *pipeline* na *package* dos controllers. Estes componentes encontram-se explicitados na Tabela 4.1.

Tabela 4.1: Responsabilidades dos componentes do pipeline de execução no Backend.

Componente	Breve Descrição da Responsabilidade
<code>AuthenticatedUserArgumentResolver</code>	Injeta automaticamente o objeto <code>AuthenticatedUser</code> nos parâmetros dos <i>controllers</i> que requerem a identidade do utilizador.
<code>AuthenticationInterceptor</code>	Intercepta os pedidos ao nível do Spring MVC para validar o <i>token</i> (<i>header</i> ou <i>cookie</i>) antes que estes atinjam os endpoints protegidos.
<code>BearerTokenAuthFilter</code>	Filtro de baixo nível que valida as credenciais do utilizador e integra a sessão ativa no contexto global de segurança do Spring Security.
<code>GlobalExceptionHandler</code>	Centraliza a captura de erros não tratados na API, transformando exceções inesperadas em respostas formatadas e seguras para o cliente.
<code>OAuthRedirectFilter</code>	Preserva e armazena na sessão o URI de redirecionamento original do utilizador durante o início do fluxo de autenticação por OAuth2.0.
<code>RequestTokenProcessor</code>	Abstrai a lógica de extração, parsing e validação de <i>tokens</i> , gerindo também a criação e destruição de <i>cookies</i> seguros.

4.1.2 Services

A camada dos serviços é onde se processa a lógica de negócio da aplicação. Esta camada está organizada de forma semelhante à camada de *controllers* já descrita, com a separação em oito serviços distintos. Estes serviços correspondem a cada um dos controllers, com exceção da ausência de um serviço para as operações invocadas pelo *Admin controller*. Neste caso, a lógica de negócio é delegada ao serviço apropriado de *User*, *Team* ou *Repo*, consoante o âmbito da operação invocada.

Um dos detalhes que vale a pena realçar é o uso que cada um dos serviços faz de uma classe que lhe é injetada com o nome de **transactionManager**. Trata-se de uma classe implementada com o nome **JdbiTransactionManager** e tem o propósito de gerir a execução de operações à base de dados dentro de transações na camada de serviços, utilizando a biblioteca JDBI, para que não seja esta camada a ter de lidar diretamente com a abertura, commit ou rollback de transações.

Tal é feito através do método *inTransaction* da JDBI, que recebe um bloco de código e executa-a dentro de uma transação controlada por essa biblioteca. Se a execução do bloco for bem-sucedida a transação é confirmada automaticamente. Caso contrário, é feito o *rollback*

também automaticamente.

4.1.3 Repositories e REST Clients

Nesta camada é onde se processa o acesso à base de dados e a APIs externas.

A package **restclient** contém as classes **GitHubRestClient** e **GitLabRestClient** onde, através do RestClient nativo do Spring, fazemos as chamadas a estas APIs. A package **repository** contém as classes onde é acedida a base de dados através de transações JDBI.

Devido à heterogeneidade dos dados, em muitas transações é necessário mapear o *ResultSet* obtido através do uso de *Data Transfer Objects* (DTOs) por nós implementados. Estes encontram-se na package **model**, junto às respetivas classes de destino.

4.1.4 Security Config

A classe **SecurityConfig** é responsável pela configuração global da segurança da aplicação através do **Spring Security**. É através desta classe que são definidos como os pedidos HTTP são autenticados, que endpoints são de acesso público ou protegidos e a integração de mecanismos de autenticação.

A anotação **@EnableWebSecurity** ativa o suporte ao **Spring Security**, permitindo a definição de regras de autenticação e autorização, bem como a integração com mecanismos como filtros personalizados e com o protocolo **OAuth2.0**.

Os métodos anotados com **@Bean** são registados no contexto do Spring como beans por ele geridos, permitindo assim que componentes como o **SecurityFilterChain** e o **AuthenticationSuccessHandler** sejam instanciados e geridos automaticamente pela framework e injetados onde necessário. Sem esse registo no *startup* da aplicação não seria possível integrar a lógica de autenticação por nós desenvolvida com recurso ao **BearerTokenAuthFilter** e ao **OAuthRedirectFilter** anteriormente discutidos no presente capítulo.

4.1.5 Normalização dos dados provenientes das APIs externas

Devido à forma distinta como as APIs do GitHub e GitLab organizam e disponibilizam as informações pertinentes ao âmbito do **SecDash** foi necessário tomar alguns cuidados para os normalizar de forma a serem agregados no domínio único da aplicação.

O primeiro passo deste processo passou por comparar as informações retornadas por cada uma das APIs e selecionar as informações relevantes para o propósito do projeto. Verificou-se que, apesar de ambas as APIs dizerem respeito a áreas semelhantes, estas não respondiam com as mesmas informações. Para resolver este problema, em primeiro lugar procedeu-se à identificação dos campos equivalentes de cada uma das APIs de cada plataforma.

Após isso foram criados *Data Transfer Objects* específicos para cada um dos recursos com anotações para identificar corretamente os campos equivalentes. Estes DTOs são então mapeados para as classes do domínio do **SecDash**.

Este processo teve de ser repetido para todos os recursos obtidos das APIs externas, nomeadamente repositórios, vulnerabilidades, alertas SAST e utilizadores das plataformas externas.

4.1.6 Pedido e *scheduling* de *scans*

As listas de vulnerabilidades podem ser obtidas pelos utilizadores sempre que estes as requisitarem através da página do repositório no *frontend*. Nesta, os utilizadores têm a possibilidade de requisitar um relatório à plataforma onde o repositório monitorizado se encontra hospedado.

Para além disso, o *backend* possui um *scheduler* para monitorização automática das alterações do número de vulnerabilidades dos repositórios de uma equipa ao longo do tempo. Este *scheduler* é um componente Spring anotado com a anotação `@Scheduled(fixedDelay = 5 * 60 * 1000)`, garantindo a sua execução a cada cinco minutos.

De forma a garantir que não existe nenhum *bottleneck* durante a execução, o *scheduler* vai apenas atualizar no máximo dez equipas em cada execução (caso estas não tenham tido atualizações nas últimas 24 horas, o seu Team Leader possuir um *token* válido (isto é, utilizado nas últimas 48 horas) armazenado na base de dados do **SecDash**). Dentro de cada equipa é então executado um pedido para cada um dos serviços ativos (Dependabot, etc) em cada repositório pertencente à equipa.

É de realçar que, com a atual implementação, o *scheduler* tem um limite de 2880 que podem ser atualizadas diariamente, que para o escopo académico do nosso projeto consideramos satisfatório e apropriado.

4.2 Frontend

A arquitetura do cliente web foi desenvolvida recorrendo à biblioteca **React**, com um foco na modularidade e reatividade da interface de utilizador.

O cliente segue uma estrutura *Single Page Application* (SPA) onde inicialmente é carregada uma única página base cujo conteúdo é atualizado dinamicamente em resposta às ações do utilizador. É assim garantida uma experiência de utilização fluida e desacoplada, com o browser a comunicar de forma assíncrona com a API do *backend* através de pedidos HTTP realizados através da função *fetch*. Quando é necessário a gestão de estado na navegação entre páginas, esta é feita através do recurso ao parâmetro do *State* do *useNavigate* do React.

As subsecções seguintes abordam alguns elementos do cliente Web que julgamos merecerem algum em destaque.

RequireAuthentication

O componente **RequireAuthentication** tem como responsabilidade interceptar os acessos a qualquer URI da aplicação que exija que o utilizador tenha sessão iniciada para que a presença de uma sessão ativa possa ser validada antes de os componentes serem exibidos. Ao tentar aceder a uma rota protegida, caso o utilizador não possua um token válido, este será automaticamente redirecionado para a página de login.

ThemeProvider

O **ThemeProvider** é o componente responsável por gerir e persistir o esquema de cores da aplicação, contando com dois temas: escuro e claro.

É garantida uma experiência visual consistente ao longo de toda a *Single Page Application*. Para evitar o efeito visual desagradável de possivelmente carregar inicialmente o tema claro e só depois ser carregado o escuro que um utilizador tenha ativado ao mudar de página, o estado inicial do tema é determinado através de uma função de inicialização *lazy* no **useState**, que verifica qual o tema selecionado pelo utilizador, armazenado no *localStorage*.

O tema atual é disponibilizado a todo o frontend através do **ThemeContext**.

4.2.1 *Internationalization* (i18n)

O frontend do **SecDash** foi implementado tendo em vista a possibilidade de exibir uma interface com várias línguas distintas.

A lógica de implementação desta funcionalidade está centrada no componente **I18nProvider**, que recorre à *Context API* do React para armazenar o idioma ativo (*Locale*) e expor o seu dicionário de traduções.

4.2.2 *Dashboard*

A principal característica do **SecDash**, para além da agregação dos relatórios de segurança oriundos de repositórios de código externos, é a exibição de gráficos no *frontend* que proporcionam uma visão rápida do estado de segurança de cada repositório individual ou conjunto de repositórios de uma equipa. Estes gráficos são gerados com recurso à biblioteca **Recharts** [18]. Cada uma destas duas vistas apresenta dois tipos de gráficos para relatórios de *Dependabot* e alertas SAST, para um total de quatro gráficos disponíveis em cada página.

Para além destes gráficos, o cliente web disponibiliza também as vistas padrão para este tipo de frontend, com vistas onde são listadas todas as vulnerabilidades de um repositório ou os membros de cada equipa, entre outros.

Estado de segurança atual

Na aba de "Visão geral" o *dashboard* apresenta gráficos circulares onde é possível verificar a distribuição de vulnerabilidades de dependências e de alertas SAST por vulnerabilidade, codificada por cores: Crítico em vermelho, Alto em laranja, Médio em amarelo e Baixo em verde). Este agrupamento por severidades é realizado pelo *backend*.

Neste painel, ilustrado na figura exibida abaixo, é também exibida a contagem de vulnerabilidades em aberto, resolvidas e ignoradas. Ao clicar num dos setores do gráfico o utilizador navega para a página onde estas vulnerabilidades são listadas, automaticamente filtrada para exibir apenas as vulnerabilidades com a severidade correspondente ao setor onde o utilizador clicou.

Evolução do estado de segurança:

Ao alternar para a aba "Histórico" o utilizador pode consultar a evolução do número de vulnerabilidades na equipa ou repositório ao longo do tempo, representada visualmente através

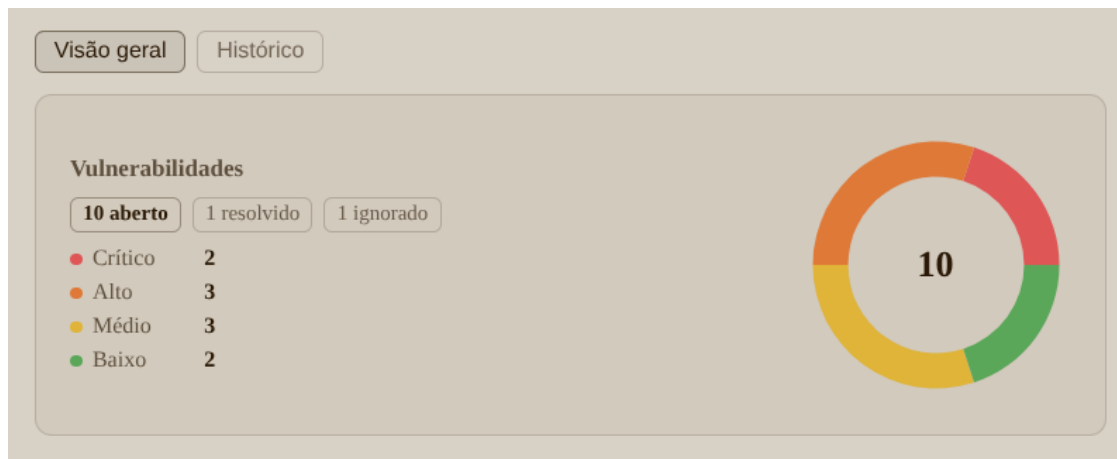


Figura 4.1: Gráfico circular para vulnerabilidades de dependências

de um gráfico de linhas. Deste modo, é possível obter de relance as vulnerabilidades que têm vindo a ser descobertas e corrigidas num repositório ou conjunto de repositórios ao longo do tempo.

Este gráfico de linhas encontra-se ilustrado na Figura 4.2, exibida abaixo. Ao clicar nas severidades as linhas correspondentes são ocultadas ou exibidas dinamicamente.

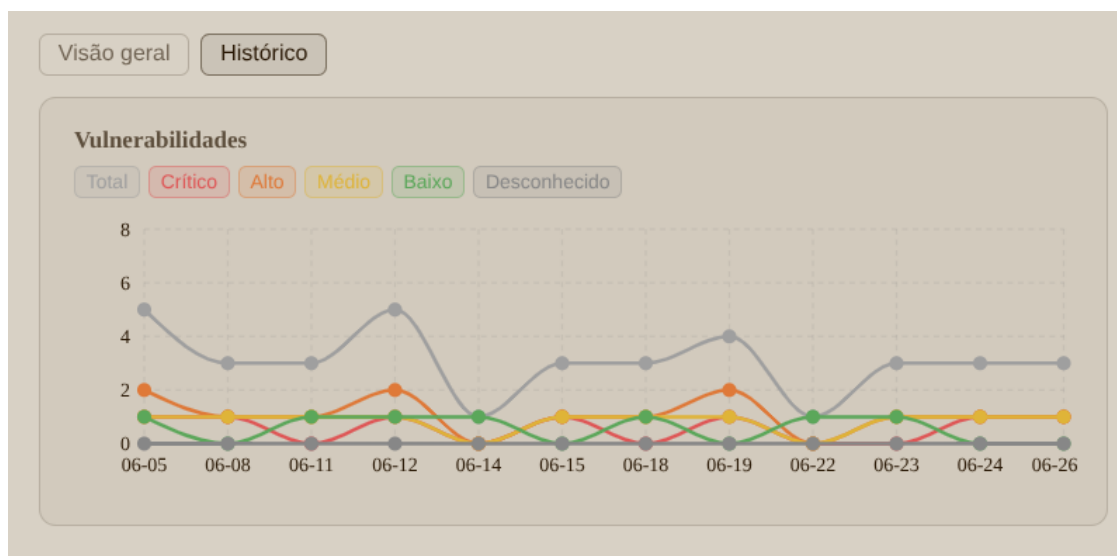


Figura 4.2: Gráfico de linhas com vulnerabilidades ao longo do tempo de um repositório

Painéis de ordenação de risco:

Na vista de equipas, por baixo dos cartões com os gráficos, o dashboard exibe outros dois cartões onde são listados, por ordem decrescente, os repositórios com maior número de vulnerabilidades de dependências e alertas SAST. Desta forma é possível verificar rapidamente

quais os repositórios mais críticos entre cada equipa e navegar rapidamente para estes. A ordenação é ponderada, com uma vulnerabilidade mais severa a ter um peso maior que uma vulnerabilidade menos crítica.

Na Figura 4.3, exibida abaixo, é possível ver um exemplo deste painel. Ao clicar no nome de um dos repositórios da lista, o utilizador navega para a página correspondente a este.



Maior risco · Vulnerabilidades		
1	web-app	● 1 ● 1 ● 1 3 total
2	payment-api	● 1 ● 1 ● 1 3 total
3	mobile-client	● 1 ● 1 ● 1 3 total
4	infra-tools	● 1 1 total

Figura 4.3: Repositórios com maior número de vulnerabilidades para uma equipa de exemplo

Capítulo 5

Conclusão

O desenvolvimento do **SecDash** foi um desafio recompensante que nos permitiu colocar em prática grande parte dos conhecimentos adquiridos ao longo da licenciatura. O seu desenvolvimento foi em si um processo de aprendizagem, colocando-nos em contacto com conceitos com os quais não estávamos tão familiarizados antes, ou com os quais não tínhamos mesmo tido qualquer contacto.

5.1 Reflexão sobre o desenvolvimento

Um dos aspetos mais positivos do projeto foi a escolha da *stack* adotada. A escolha de React, Spring Boot e PostgreSQL, pelas suas funcionalidades bem como pela nossa experiência com estas, facilitou o desenvolvimento do projeto, garantindo um bom ritmo de trabalho sem bloqueios significativos ao longo da implementação. Não obstante, deparámo-nos com alguns desafios, de entre os quais:

- **Desenho e Evolução do Modelo de Dados:** Uma das maiores dificuldades sentidas prendeu-se pelo desenho inicial do modelo da base de dados. Algumas funcionalidades, tal como a necessidade de armazenar o histórico de vulnerabilidades ao longo do tempo, não foram devidamente ponderadas durante a fase de planeamento inicial do projeto. Isto resultou na refatoração do modelo de dados por várias vezes, tendo como consequência uma considerável perda de tempo que talvez pudesse ter sido evitada caso o modelo de dados tivesse sido planeado de forma mais refletida desde o início.
- **Gestão de Autenticação e Segurança:** Para a gestão de sessões, optou-se por desenhar um sistema próprio de validação e gestão de *tokens*. Em retrospectiva, esta decisão introduziu uma complexidade desnecessária que talvez pudesse ter sido evitada ao explorar em maior profundidade o ecossistema do **Spring Security** e delegando-lhe toda a gestão de autenticação.
- **Análise e Exposição de Vulnerabilidades (CVEs):** Ao longo do desenvolvimento

do projeto, o nosso conhecimento em torno do ecossistema de *Common Vulnerabilities and Exposures* (CVE), apesar de se ter aprofundado, manteve-se algo abstrato. Uma investigação mais aprofundada sobre a natureza destas métricas poderia ter permitido desenhar painéis informativos mais pertinentes para um utilizador da aplicação focado na segurança dos seus repositórios.

- **CSS e Design:** Com o intuito de consolidar conhecimentos neste campo, a abordagem inicial de design do *frontend* passou por codificar todo o CSS manualmente. Contudo, à medida que a complexidade do cliente web cresceu, esta decisão revelou-se insustentável em termos de produtividade, tendo-se eventualmente recorrido a agentes de Inteligência Artificial para auxiliar o desenvolvimento de uma interface visualmente apelativa e agradável. Assim sendo, a adoção de uma *framework* como Bootstrap ou o Tailwind CSS talvez tivesse sido uma melhor opção.
- **Design Responsivo:** O **SecDash** foi projetado com um foco em ecrãs de computador (*desktop-first*). Consequentemente, as páginas web e as visualizações gráficas não se encontram otimizadas para dispositivos móveis, o que limita a acessibilidade e a usabilidade da plataforma em telemóveis ou *tablets*.

5.2 Trabalho Futuro

Tendo em conta as limitações com que nos deparamos ao longo do desenvolvimento, consideramos que faria sentido explorar as seguintes melhorias em iterações futuras do projeto:

- **Refatoração da gestão de *tokens*:** Substituir o sistema de *tokens* implementado por nós e delegar toda a parte de autenticação e gestão de sessões puramente ao **Spring Security**, simplificando assim a arquitetura do nosso *backend*.
- **Known Exploited Vulnerabilities:** A possibilidade de cruzar os relatórios de segurança da plataforma com o catálogo KEV (*Known Exploited Vulnerabilities*) permitiria colocar em destaque no *dashboard* quais as vulnerabilidades que estão ativamente a ser exploradas em ataques no mundo real, ajudando a priorizar as vulnerabilidades de forma muito mais pragmática.
- **Otimização de Interface Mobile:** Adaptar o CSS do *frontend* e reestruturar alguns dos componentes gráficos para os tornar responsivos, de modo a garantir que a plataforma tenha uma experiência de utilização e acessibilidade em telemóveis e *tablets* comparável à da sua utilização num computador.

Referências

- [1] D. L. Nguyen, D. H. Phan, T. D. Vu, and D. T. Ta, “Risk management in projects based on open-source software,” in *Proceedings of the 2019 8th International Conference on Software and Computer Applications (ICSCA '19)*, February 2019, pp. 178–183. [Online]. Available: <https://dl.acm.org/doi/10.1145/3316615.3316648>
- [2] CSIS — Center for Strategic and International Studies. (2024) Significant cyber incidents. Strategic Technologies Program. Acedido em: 04-03-2026. [Online]. Available: <https://www.csis.org/programs/strategic-technologies-program/significant-cyber-incidents>
- [3] CVE Program. (2014) CVE-2014-0160: Heartbleed vulnerability in openssl. MITRE Corporation. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2014-0160>
- [4] National Institute of Standards and Technology. (2021) CVE-2021-44228: Apache log4j2 remote code execution vulnerability. National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>
- [5] L. H. Newman. (2021) The Log4Shell vulnerability is just the beginning. Conde Nast. [Online]. Available: <https://www.wired.com/story/log4j-log4shell-vulnerability-ransomware-second-wave/>
- [6] D. Goodin. (2021) As Log4Shell wreaks havoc, payroll service reports ransomware attack. Condé Nast. [Online]. Available: <https://arstechnica.com/information-technology/2021/12/as-log4shell-wreaks-havoc-payroll-service-reports-ransomware-attack/>
- [7] R. Lerman and C. Zakrzewski. (2021) The Log4j vulnerability has the internet on high alert. [Online]. Available: <https://www.washingtonpost.com/technology/2021/12/20/log4j-hack-vulnerability-java/>
- [8] United States Senate, “How Equifax failed to prevent the 2017 data breach,” Permanent Subcommittee on Investigations, Committee on Homeland Security and Governmental Affairs, Tech. Rep., 2019, staff Report. [Online]. Available: <https://nsarchive.gwu.edu/document/18370-national-security-archive-u-s-senate-permanent>

- [9] EPIC — Electronic Privacy Information Center. (2017) The Equifax data breach. Public Interest Research Center. [Online]. Available: <https://archive.epic.org/privacy/data-breach/equifax/>
- [10] SonarSource. (2026) What is SAST? Static Application Security Testing Definition & Guide. Acedido em: 24 de Maio de 2026. [Online]. Available: <https://www.sonarsource.com/resources/library/sast/>
- [11] GitLab. (2026) Static Application Security Testing (SAST) documentation. Acedido em: 24 de Maio de 2026. [Online]. Available: https://docs.gitlab.com/user/application_security/sast/
- [12] DefectDojo, “Defectdojo – devsecops and vulnerability management platform,” 2026, acedido em: Maio de 2026. [Online]. Available: <https://defectdojo.com/>
- [13] ArmorCode, “Armorcode – the market leader in next-gen aspm,” 2026, acedido em: Maio de 2026. [Online]. Available: <https://www.armorcode.com/>
- [14] DefectDojo Project. (2024) About DefectDojo: DevSecOps, ASOC, and Vulnerability Management. Official Documentation. [Online]. Available: https://docs.defectdojo.com/get_started/about/about_defectdojo/
- [15] ——. (2024) Product hierarchy: Product Types, Products, and Engagements. DefectDojo Documentation - Asset Modelling. [Online]. Available: https://docs.defectdojo.com/asset_modelling/hierarchy/product_hierarchy/
- [16] ArmorCode Inc. (2024) Unified vulnerability management: A modern approach. Official Platform Overview. [Online]. Available: <https://www.armorcode.com/unified-vulnerability-management>
- [17] Spring Framework Contributors. (2026) Spring framework overview. Broadcom / Spring. [Online]. Available: <https://docs.spring.io/spring-framework/reference/overview.html>
- [18] Recharts Group, “Recharts: A composable charting library built on react components,” 2026, acedido a: 5 de julho de 2026. [Online]. Available: <https://recharts.github.io/>

Apêndice A

Exemplo do formato de resposta JSON de um repositório GitHub

```
1      {
2          "id": 735375108,
3          "node_id": "R_kgDOK9TvBA",
4          "name": "IPW-FINAL",
5          "full_name": "gfdcarvalho/IPW-FINAL",
6          "private": true,
7          "owner": {
8              "login": "gfdcarvalho",
9              "id": 87329708,
10             "node_id": "MDQ6VXNlcjg3MzI5NzA4",
11             "avatar_url": "https://avatars.githubusercontent.com/u/87329708?v=4",
12             "gravatar_id": "",
13             "url": "https://api.github.com/users/gfdcarvalho",
14             "html_url": "https://github.com/gfdcarvalho",
15             "followers_url": "https://api.github.com/users/gfdcarvalho/followers",
16             "following_url": "https://api.github.com/users/gfdcarvalho/following{/other_user}",
17             "gists_url": "https://api.github.com/users/gfdcarvalho/gists{/gist_id}",
18             "starred_url": "https://api.github.com/users/gfdcarvalho/starred{/owner}/{/repo}",
19             "subscriptions_url": "https://api.github.com/users/gfdcarvalho/subscriptions",
20             "organizations_url": "https://api.github.com/users/gfdcarvalho/orgs",
21             "repos_url": "https://api.github.com/users/gfdcarvalho/repos",
22             "events_url": "https://api.github.com/users/gfdcarvalho/events{/privacy}",
23             "received_events_url": "https://api.github.com/users/gfdcarvalho/received_events",
24             "type": "User",
25             "user_view_type": "public",
26             "site_admin": false
27         },
```

```

28     "html_url": "https://github.com/gfdcarvalho/IPW-FINAL
29     ",
30     "description": null,
31     "fork": false,
32     "url": "https://api.github.com/repos/gfdcarvalho/IPW-
33     FINAL",
34     "forks_url": "https://api.github.com/repos/gfdcarvalho
35     /IPW-FINAL/forks",
36     "keys_url": "https://api.github.com/repos/gfdcarvalho/
37     IPW-FINAL/keys{/key_id}",
38     "collaborators_url": "https://api.github.com/repos/
39     gfdcarvalho/IPW-FINAL/collaborators{/collaborator
40     }",
41     "teams_url": "https://api.github.com/repos/gfdcarvalho
42     /IPW-FINAL/teams",
43     "hooks_url": "https://api.github.com/repos/gfdcarvalho
44     /IPW-FINAL/hooks",
45     "issue_events_url": "https://api.github.com/repos/
46     gfdcarvalho/IPW-FINAL/issues/events{/number}",
47     "events_url": "https://api.github.com/repos/
48     gfdcarvalho/IPW-FINAL/events",
49     "assignees_url": "https://api.github.com/repos/
50     gfdcarvalho/IPW-FINAL/assignees{/user}",
51     "branches_url": "https://api.github.com/repos/
52     gfdcarvalho/IPW-FINAL/branches{/branch}",
53     "tags_url": "https://api.github.com/repos/gfdcarvalho/
54     IPW-FINAL/tags",
55     "blobs_url": "https://api.github.com/repos/gfdcarvalho
56     /IPW-FINAL/git/blobs{/sha}",
57     "git_tags_url": "https://api.github.com/repos/
58     gfdcarvalho/IPW-FINAL/git/tags{/sha}",
59     "git_refs_url": "https://api.github.com/repos/
60     gfdcarvalho/IPW-FINAL/git/refs{/sha}",
61     "trees_url": "https://api.github.com/repos/gfdcarvalho
62     /IPW-FINAL/git/trees{/sha}",
63     "statuses_url": "https://api.github.com/repos/
64     gfdcarvalho/IPW-FINAL/statuses/{sha}",
65     "languages_url": "https://api.github.com/repos/
66     gfdcarvalho/IPW-FINAL/languages",
67     "stargazers_url": "https://api.github.com/repos/
68     gfdcarvalho/IPW-FINAL/stargazers",
69     "contributors_url": "https://api.github.com/repos/
70     gfdcarvalho/IPW-FINAL/contributors",
71     "subscribers_url": "https://api.github.com/repos/
72     gfdcarvalho/IPW-FINAL/subscribers",
73     "subscription_url": "https://api.github.com/repos/
74     gfdcarvalho/IPW-FINAL/subscription",
75     "commits_url": "https://api.github.com/repos/
76     gfdcarvalho/IPW-FINAL/commits{/sha}",
77     "git_commits_url": "https://api.github.com/repos/
78     gfdcarvalho/IPW-FINAL/git/commits{/sha}",
79     "comments_url": "https://api.github.com/repos/
80     gfdcarvalho/IPW-FINAL/comments{/number}",
81     "issue_comment_url": "https://api.github.com/repos/
82     gfdcarvalho/IPW-FINAL/issues/comments{/number}",

```

```

56 "contents_url": "https://api.github.com/repos/
57   gfdcarvalho/IPW-FINAL/contents/{+path}",
58 "compare_url": "https://api.github.com/repos/
59   gfdcarvalho/IPW-FINAL/compare/{base}...{head}",
60 "merges_url": "https://api.github.com/repos/
61   gfdcarvalho/IPW-FINAL/merges",
62 "archive_url": "https://api.github.com/repos/
63   gfdcarvalho/IPW-FINAL/{archive_format}/{ref}",
64 "downloads_url": "https://api.github.com/repos/
65   gfdcarvalho/IPW-FINAL/downloads",
66 "issues_url": "https://api.github.com/repos/
67   gfdcarvalho/IPW-FINAL/issues{/number}",
68 "pulls_url": "https://api.github.com/repos/gfdcarvalho
69   /IPW-FINAL/pulls{/number}",
70 "milestones_url": "https://api.github.com/repos/
71   gfdcarvalho/IPW-FINAL/milestones{/number}",
72 "notifications_url": "https://api.github.com/repos/
73   gfdcarvalho/IPW-FINAL/notifications{?since,all,
74   participating}",
75 "labels_url": "https://api.github.com/repos/
76   gfdcarvalho/IPW-FINAL/labels{/name}",
77 "releases_url": "https://api.github.com/repos/
78   gfdcarvalho/IPW-FINAL/releases{/id}",
79 "deployments_url": "https://api.github.com/repos/
80   gfdcarvalho/IPW-FINAL/deployments",
81 "created_at": "2023-12-24T17:26:14Z",
82 "updated_at": "2023-12-24T17:28:00Z",
83 "pushed_at": "2023-12-29T00:01:03Z",
84 "git_url": "git://github.com/gfdcarvalho/IPW-FINAL.git",
85 "ssh_url": "git@github.com:gfdcarvalho/IPW-FINAL.git",
86 "clone_url": "https://github.com/gfdcarvalho/IPW-FINAL
87   .git",
88 "svn_url": "https://github.com/gfdcarvalho/IPW-FINAL",
89 "homepage": null,
90 "size": 4045,
91 "stargazers_count": 0,
92 "watchers_count": 0,
93 "language": "JavaScript",
94 "has_issues": true,
95 "has_projects": true,
96 "has_downloads": true,
97 "has_wiki": false,
98 "has_pages": false,
99 "has_discussions": false,
100 "forks_count": 0,
101 "mirror_url": null,
102 "archived": false,
103 "disabled": false,
104 "open_issues_count": 0,
105 "license": null,
106 "allow_forking": true,
107 "is_template": false,
108 "web_commit_signoff_required": false,
109 "has_pull_requests": true,
110 "pull_request_creation_policy": "all",

```

```
97         "topics": [],
98         "visibility": "private",
99         "forks": 0,
100        "open_issues": 0,
101        "watchers": 0,
102        "default_branch": "main",
103        "permissions": {
104            "admin": false,
105            "maintain": false,
106            "push": true,
107            "triage": true,
108            "pull": true
109        }
110    }
```

Apêndice B

Script SQL para a criação da base de dados

```
1
2
3      CREATE TYPE platform AS ENUM ('GITHUB', 'GITLAB');
4      CREATE TYPE visibility AS ENUM ('PUBLIC', 'PRIVATE', 'INTERNAL
5      ');
6      CREATE TYPE vulnerability_severity AS ENUM ('CRITICAL', 'HIGH',
7      ', 'MEDIUM', 'LOW', 'UNKNOWN');
8      CREATE TYPE vulnerability_state AS ENUM ('OPEN', 'FIXED', '
9      DISMISSED');
10     CREATE TYPE sast_severity AS ENUM ('CRITICAL', 'HIGH', 'MEDIUM
11     ', 'LOW', 'UNKNOWN');
12     CREATE TYPE sast_state AS ENUM ('OPEN', 'FIXED', 'DISMISSED');
13     CREATE TYPE auth_provider AS ENUM ('GOOGLE', 'GITHUB', 'GITLAB
14     ');
15     CREATE TYPE app_roles as ENUM ('ADMIN', 'USER');
16     CREATE TYPE team_roles as ENUM ('LEADER', 'COLLABORATOR');
17
18     CREATE TABLE users (
19         uid                INT GENERATED ALWAYS AS IDENTITY
20         PRIMARY KEY,
21         name                VARCHAR(255) NOT NULL,
22         password_validation TEXT,
23         email               VARCHAR(255) NOT NULL UNIQUE,
24         role                app_roles NOT NULL DEFAULT 'USER'
25     );
26
27     CREATE TABLE user_authentication (
28         user_id            INT NOT NULL REFERENCES users(uid),
29         provider            auth_provider NOT NULL,
30         provider_id         VARCHAR NOT NULL,
31         PRIMARY KEY (user_id, provider),
32         UNIQUE              (provider, provider_id)
33     );
34
35     CREATE TABLE user_authorization (
36         user_id            INT NOT NULL REFERENCES users(uid),
37         provider            auth_provider NOT NULL,
```

```

34      access_token  TEXT          NOT NULL,
35      refresh_token TEXT,
36      expires_at    TIMESTAMPTZ,
37      PRIMARY KEY   (user_id, provider)
38  );
39
40
41  CREATE TABLE tokens (
42      token_validation VARCHAR(256) primary key,
43      user_id          int references users(uid),
44      created_at       bigint not null,
45      last_used_at     bigint not null
46  );
47
48
49  CREATE TABLE owners (
50      oid            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
51      external_id    VARCHAR(255),
52      name           VARCHAR(255) NOT NULL,
53      url            VARCHAR(255) NOT NULL,
54      avatar_url     VARCHAR(255),
55      platform       platform NOT NULL,
56      UNIQUE (external_id, platform)
57  );
58
59
60  CREATE TABLE repositories (
61      rid            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
62      name           VARCHAR(255) NOT NULL,
63      external_id    VARCHAR(255) NOT NULL,
64      platform       platform NOT NULL,
65      owner_id       INT NOT NULL REFERENCES owners(oid),
66      html_url       VARCHAR(255) NOT NULL,
67      description    TEXT,
68      issues_count   INT NOT NULL DEFAULT 0,
69      created_at     TIMESTAMPTZ NOT NULL,
70      updated_at     TIMESTAMPTZ NOT NULL,
71      forks_count    INT NOT NULL DEFAULT 0,
72      visibility     visibility NOT NULL,
73      UNIQUE (external_id, platform)
74  );
75
76
77  CREATE TABLE vulnerabilities (
78      vid            INT GENERATED ALWAYS AS IDENTITY
79          PRIMARY KEY,
80      external_id    VARCHAR(255) NOT NULL,
81      title          VARCHAR(255) NOT NULL,
82      description    TEXT,
83      severity       vulnerability_severity NOT NULL,
84      state          vulnerability_state NOT NULL,
85      cve_id         VARCHAR(50),
86      ghsa_id        VARCHAR(50),
87      package_name   VARCHAR(255) NOT NULL,
88      package_version VARCHAR(100),
89      vulnerable_version_range VARCHAR(255),

```

```

89     fixed_version          VARCHAR(100),
90     manifest_path         VARCHAR(255),
91     cvss_score             DOUBLE PRECISION,
92     cvss_vector            VARCHAR(255),
93     platform               platform      NOT NULL,
94     rid                    INT           NOT NULL
          REFERENCES repositories(rid),
95     detected_at            TIMESTAMP     NOT NULL,
96     updated_at             TIMESTAMP     NOT NULL,
97     UNIQUE (external_id, platform, rid)
98 );
99
100
101 CREATE TABLE repo_vulnerability_scans (
102     scan_id                INT GENERATED ALWAYS AS IDENTITY PRIMARY
          KEY,
103     rid                    INT           NOT NULL REFERENCES
          repositories(rid),
104     scanned_at             TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
105     vulnerability_count    INT           NOT NULL,
106     critical_count         INT           NOT NULL DEFAULT 0,
107     high_count             INT           NOT NULL DEFAULT 0,
108     medium_count           INT           NOT NULL DEFAULT 0,
109     low_count              INT           NOT NULL DEFAULT 0,
110     unknown_count          INT           NOT NULL DEFAULT 0
111 );
112
113
114 CREATE TABLE vulnerability_references (
115     id                     INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
116     vuln_id               INT NOT NULL REFERENCES vulnerabilities(vid),
117     url                   TEXT NOT NULL
118 );
119
120
121 CREATE TABLE user_repositories (
122     uid INT NOT NULL REFERENCES users(uid),
123     rid INT NOT NULL REFERENCES repositories(rid),
124     PRIMARY KEY (uid, rid)
125 );
126
127
128 CREATE TABLE sast_alerts (
129     sid                    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
130     rid                    INT           NOT NULL REFERENCES
          repositories(rid),
131     external_id            VARCHAR(255)  NOT NULL,
132     state                  sast_state    NOT NULL,
133     severity               sast_severity NOT NULL,
134     rule_id                VARCHAR(255)  NOT NULL,
135     rule_description        TEXT          NOT NULL,
136     tool_name              VARCHAR(255)  NOT NULL,
137     file_path              VARCHAR(255),
138     start_line             INT,
139     end_line               INT,
140     message                TEXT,

```

```

141      html_url          VARCHAR(255) NOT NULL,
142      platform          platform NOT NULL,
143      detected_at       TIMESTAMPTZ,
144      updated_at        TIMESTAMPTZ,
145      UNIQUE (external_id, platform, rid)
146  );
147
148
149  CREATE TABLE repo_sast_scans (
150      scan_id           INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
151      rid               INT NOT NULL REFERENCES repositories(rid)
152  ),
153      scanned_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
154      alert_count      INT NOT NULL,
155      critical_count    INT NOT NULL DEFAULT 0,
156      high_count        INT NOT NULL DEFAULT 0,
157      medium_count      INT NOT NULL DEFAULT 0,
158      low_count         INT NOT NULL DEFAULT 0,
159      unknown_count     INT NOT NULL DEFAULT 0
160  );
161
162  CREATE TABLE teams (
163      tid               INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
164      name              VARCHAR(255) NOT NULL,
165      description       TEXT,
166      last_scan_at     TIMESTAMPTZ
167  );
168
169  CREATE TABLE team_users (
170      tid              INT NOT NULL REFERENCES teams(tid),
171      uid              INT NOT NULL REFERENCES users(uid),
172      role             team_roles NOT NULL,
173      PRIMARY KEY (tid, uid)
174  );
175
176  CREATE TABLE team_repos (
177      tid              INT NOT NULL REFERENCES teams(tid),
178      rid              INT NOT NULL REFERENCES repositories(rid),
179      PRIMARY KEY (tid, rid)
180  );

```